04 - Tablas de Hashing

Diplomado Data Engineer

Universidad de Santiago de Chile

UNIVERSIDAD DE SANTIAGO DE CHILE

Juan Pablo Castillo

Problema de Búsqueda

Supongamos que tenemos una colección L de n datos de la siguiente forma:

$$L = \{(k_0, I_0), (k_1, I_1), \ldots, (k_{n-1}, I_{n-1})\}$$

- Donde I_j es la información o dato asociado a la clave k_j.
- Dada una clave en particular K, el **problema de búsqueda** consiste en localizar el registro (k_j, I_j) en L tal que $k_j = K$ (si es que existe).
- Estaremos interesados solo en búsquedas exactas.
  - i.e., K debe ser exactamente igual a k_j.

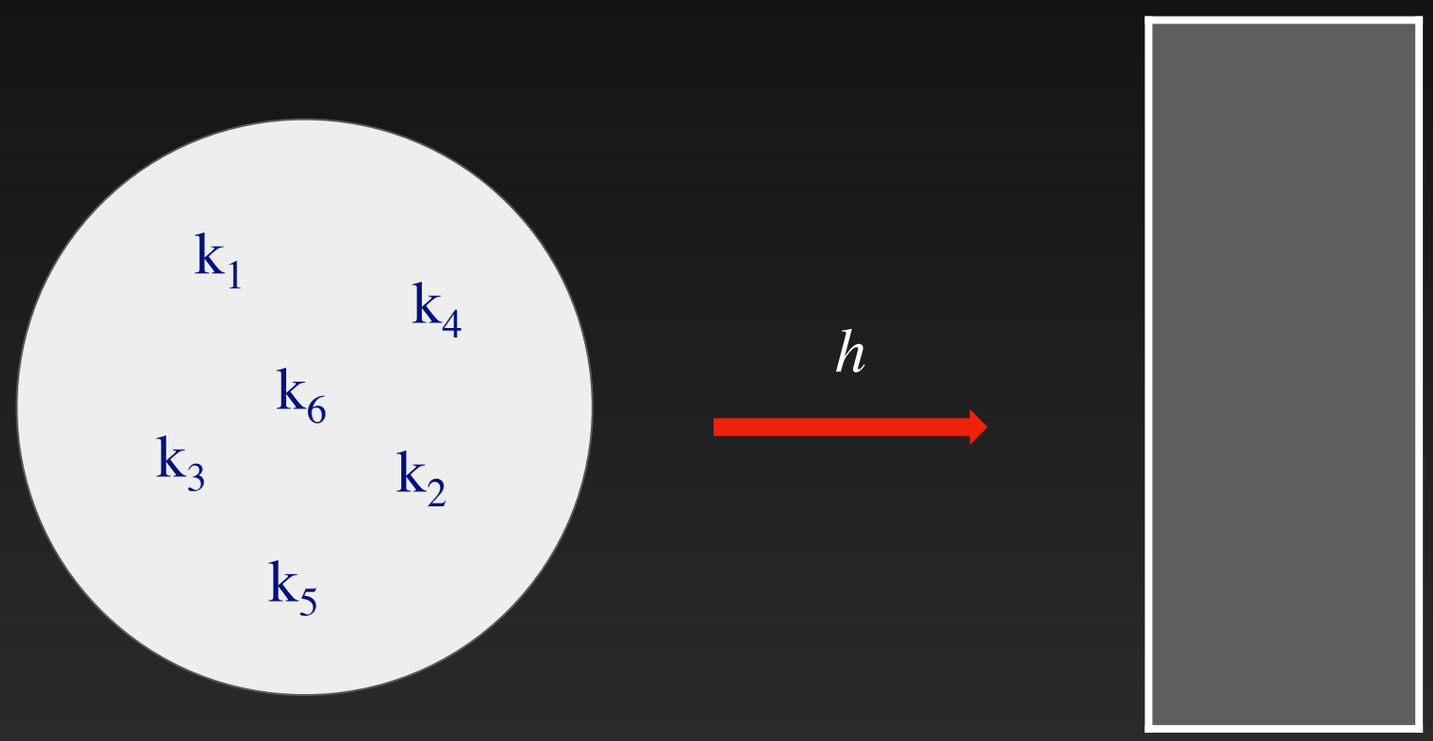
Tablas de Hashing

- Estudiamos a continuación un método para indexar arreglos que es totalmente distinto a los métodos vistos hasta ahora.
  - e.g., indexar por la posición
- Dado un espacio de memoria contigua (**un arreglo**) que almacena los elementos de un conjunto L.
  - El siguiente método proveerá **acceso directo a los elementos** de L por medio de la clave de búsqueda.
- Esto es, las claves de búsqueda serán transformadas en direcciones de memoria dentro del arreglo en donde buscar el registro buscado.
- Estas técnicas se conocen como "**hashing**" o "dispersión".

Técnicas de Hashing

Conjunto de Claves

Espacio de almacenamiento



Usaremos la dirección de memoria producida por h para almacenar los registros

La función de hashing h mapea claves a una dirección dentro del espacio de almacenamiento

Técnicas de Hashing

Conjunto de Claves

χ

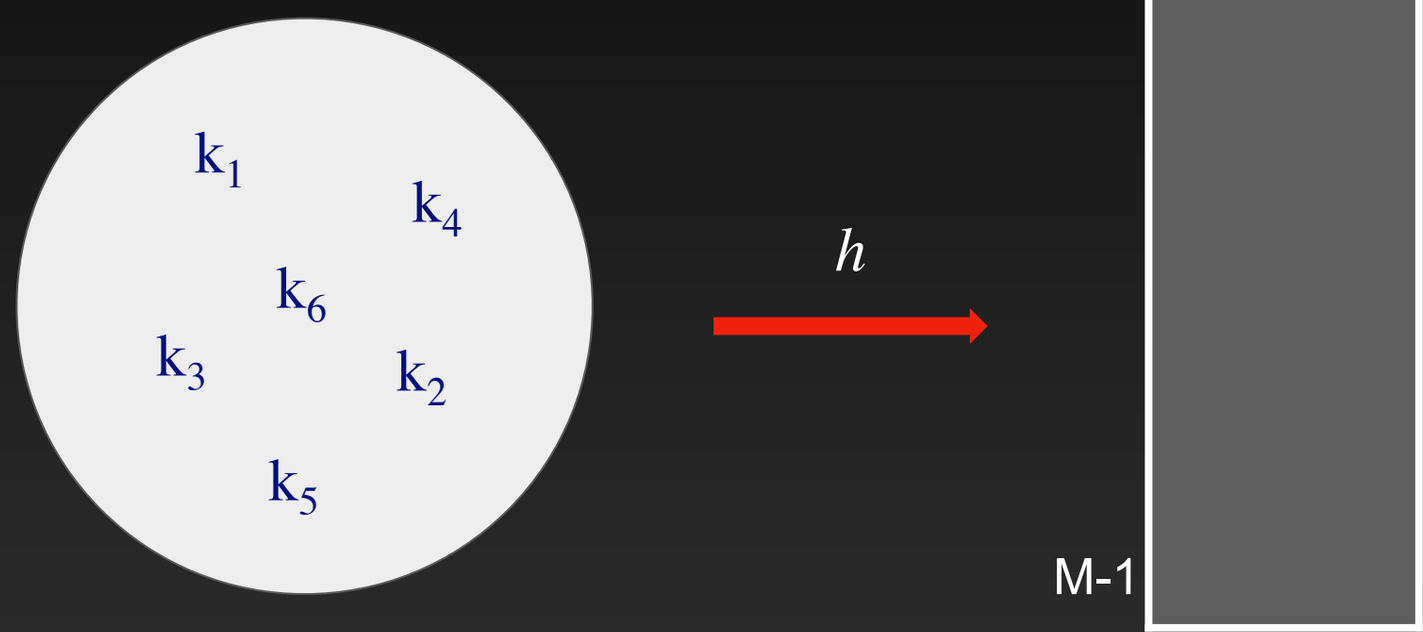


Tabla de Hashing HT

El espacio de almacenamiento está dividido en **ranuras**

Cada ranura es capaz de almacenar un **registro**

Hay M ranuras en total

$$h : X \rightarrow \{0, \ldots, M-1\}$$

Técnicas de Hashing

Idea Genera de la técnica

- Sea K una clave, e I sea su información asociada.
- Si $h(K) = i$, entonces almacenamos

$$HT[i] \leftarrow (K, I)$$

Esto tiene las siguientes vetajas:

- Facilita la inserción y eliminación rápida de registros
- Encuentra registros con muy pocos accesos al arreglo en promedio
- La idea es almacenar los registros en la forma menos relacionada posible, para que su acceso sea eficiente
    - Lo ideal es lograr una adecuada **repartición** de los datos a lo largo de toda la tabla/arreglo.

Técnicas de Hashing

Características Principales de la Técnica

- Hashing no es adecuado cuando se deben soportar claves repetidas

  Las claves tienen que ser ***únicas*** para usar hashing

- Hashing no permite recorrer de manera eficiente los elementos en algún orden dado
  - e.g., las claves ordenadas por valor
- Hashing no permite obtener el valor mínimo o máximo de manera eficiente
  - Su principal aplicación son los ***problemas de búsqueda exacta***.
  - Es simple de implementar y el diseño de funciones h está bastante estudiado y probado.
  - Eficiente tanto en memoria principal (RAM) como en memoria persistente (e.g., disco)

Técnicas de Hashing

Un ejemplo muy simple de hashing es el siguiente:

- Se tienen n datos cuyas claves pueden tener un valor $\{0,1,2,\ldots,n-1\}$
- Se tiene un arreglo $HT[0..n-1]$
- Se define $h(k) = k$, i.e., la función identidad
- Si para una clave K, tenemos que $h(K) = i$, la clave se almacenará:

$$HT[i] = (K, I)$$

- Cuando queramos buscar la clave K, aplicamos nuevamente la función h sobre K.
- Tiempo de búsqueda $O(1)$

Técnicas de Hashing

Colisiones

En la práctica la situación es muy distinta por los siguiente motivos

- Generalmente, el rango de valores que pueden tomar las claves es mucho mayor que la cantidad de datos que almacenaremos (incluso más que la cantidad de ranuras de HT)
  - i.e., $|\chi| \gg n$
- Por ejemplo, un conjunto de claves con 1000 millones de valores posibles, pero que se sabe que habrán solo 1000 registros.
- En esos casos, no se puede actuar como en el ejemplo anterior , si no que utilizamos una tabla mucho más pequeña para no desperdiciar demasiado espacio.

Técnicas de Hashing

Colisiones

Esta situación de tener menos ranuras que los posibles valores para las claves, hace que potencialmente ***claves diferentes se mapean a la misma ranura de la tabla***.

- i.e., es difícil por no decir imposible hacer que h sea inyectiva (sería lo ideal)
- Entonces, diremos que dos claves k_1 y k_2 colisionan en la ranura b bajo la función h cuando:

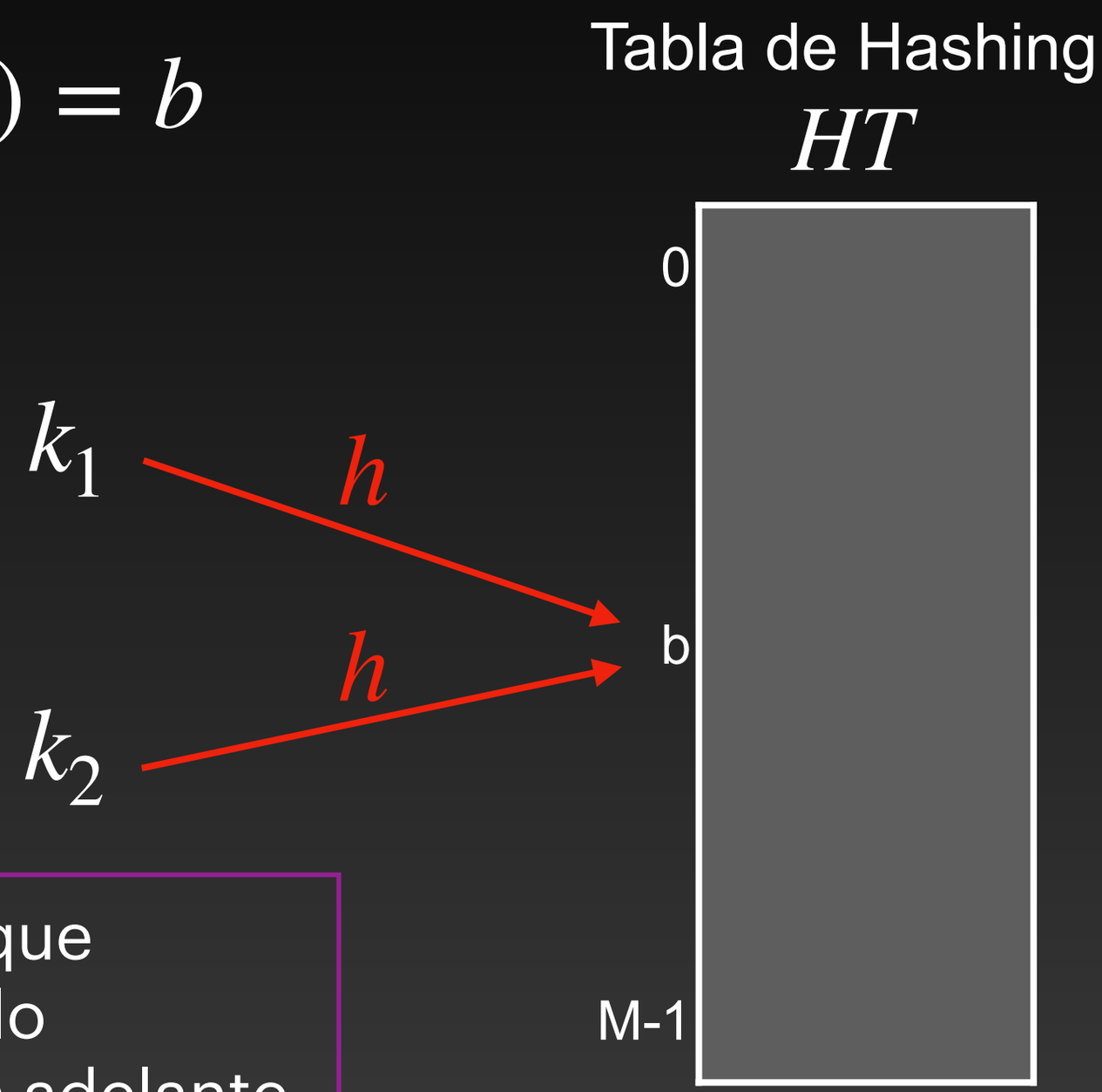
$$h(k_1) = h(k_2) = b$$

También se cono se como que k_1 y k_2 son sinónimos bajo h

Técnicas de Hashing

Colisiones

$$h(k_1) = h(k_2) = b$$



De alguna forma hay que solucionar esto, pero lo veremos un poco más adelante.

Técnicas de Hashing

Como buscar en una tabla de hashing?

Antes de ver como se pueden solucionar las colisiones en una tabla de hashing.

Para encontrar un dato con clave K en una base de datos organizada mediante una técnica de hashing, seguimos el siguiente proceso:

1. Computamos el valor $i = h(K)$,
   - I,e., calculamos la casilla en donde debería ir el dato asociado a la clave K.
2. Comenzamos de la ranura i, probamos una secuencia de ranuras de HT que siguen una ***política de resolución de colisiones***.

Ahora vemos como solucionar las colisiones.

Nota: no es necesario que entienda en profundidad lo que sigue, solo a modo de ilustración de como funcionaría una tabla de hashing.

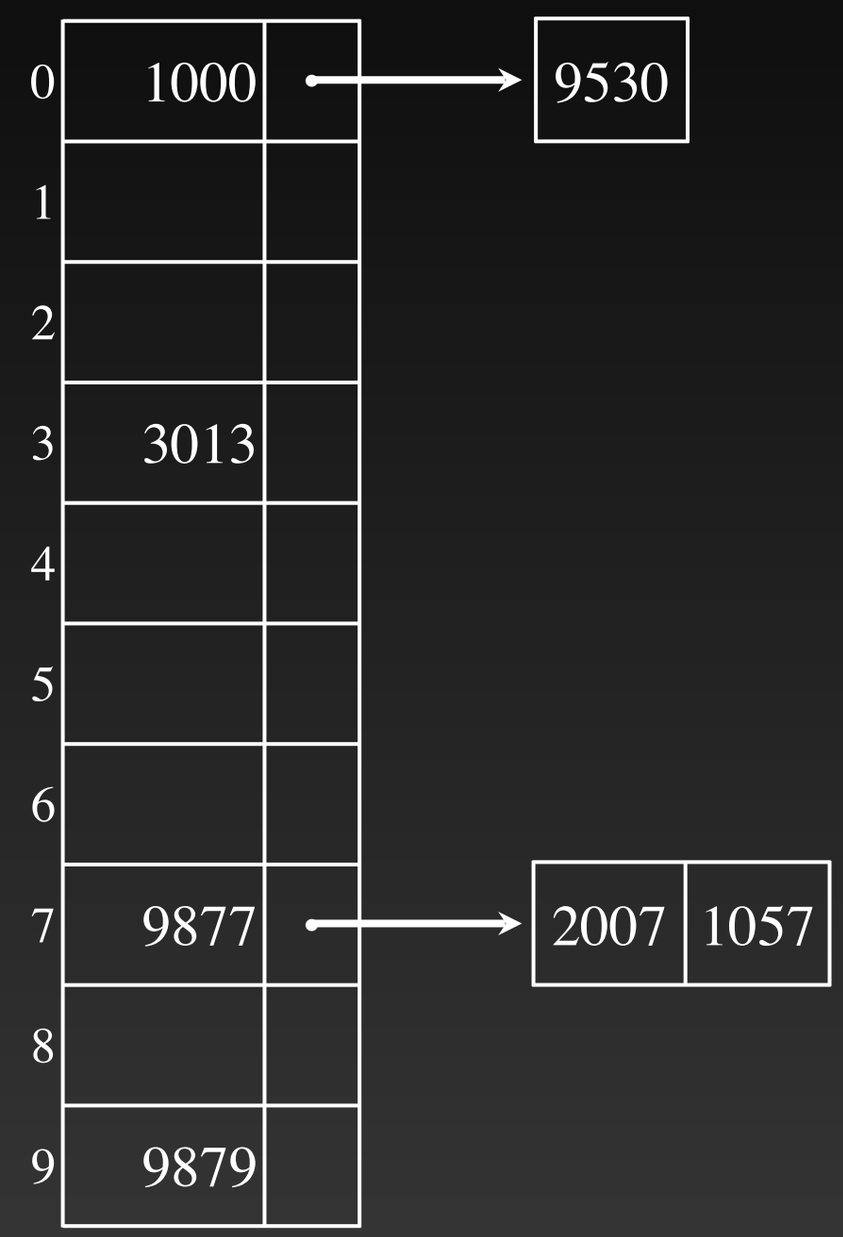
Técnicas de Resolución de Colisiones

Hashing Abierto

- Las funciones de hashing se diseñan con el objetivo de minimizar las colisiones.
- Sin embargo, las colisiones son ***inevitables*** (muy dificil encontrar funciones inyectivas)
- Por lo tanto, se deben implementar estrategias de resolución de colisiones, que cumplan adecuadamente su tarea.
- Dichas estrategias pueden dividirse en dos clases principales:
  - ***Hashing abierto:*** las colisiones se almacenan fuera de la tabla.
  - ***Hashing cerrado:*** *las colisiones se almacenan en una ranura diferente en la tabla.*

Hashing Abierto

- La técnica más simple consiste en definir a cada ranura de la tabla como la cabeza de una lista.
- Todos los registros cuya clave es mapeada a una ranura i son almacenadas en la lista de esa ranura.
- Por ejemplo, para una tabla con $M = 10$, tenemos la siguiente función h:
  - *h(1000) = h(9530) = 0*
  - *h(3013) = 3*
  - *h(9877) = h(2007) = h(1057) = 7*
  - *h(9879) = 9*



Hashing Abierto

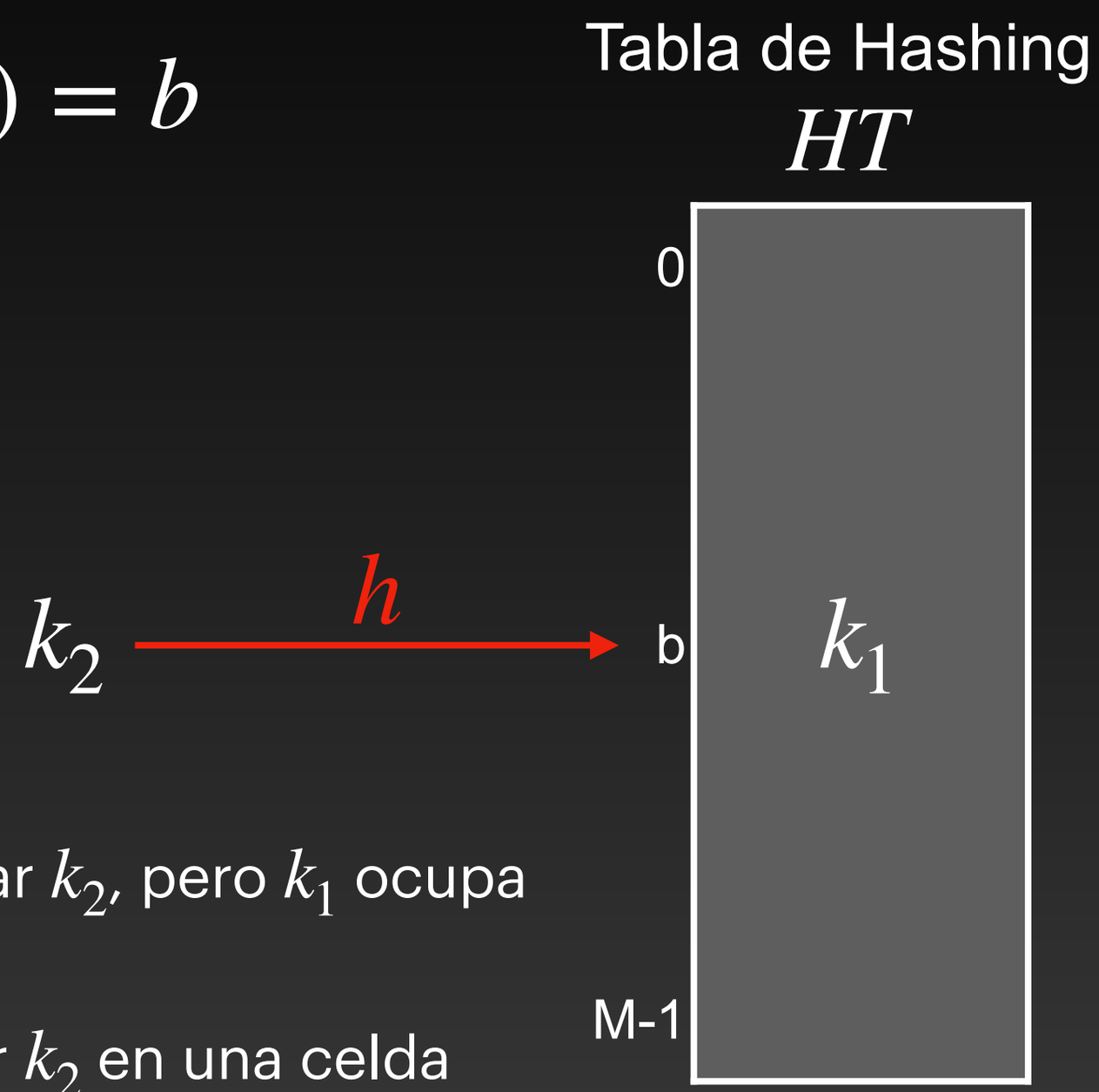
- Dentro de una lista, los registros pueden ordenarse como sea necesario:
  - Por orden de llegada, por valor de la clave, por frecuencia de acceso (los más accedidos al comienzo de la lista), etc.
- Dada una tabla de M ranuras y, n registros a ser almacenados,
  - La función h idealmente producirá listas de largo $\frac{n}{m}$ cada una
  - Si hay más ranuras que registros a almacenar, pocas ranuras deberían tener más de un registro en su lista
- Se puede llegar a tener costo de búsqueda $O(1)$, si asumimos que la función h distribuye los elementos de manera uniforme en la tabla
  - Pero si la función h no es adecuada, podemos llegar a tener costo $O(n)$

Hashing Cerrado

- Los registros se almacenan directamente dentro de la misma tabla.
- Si un registro (k, I) debe ser insertado y la ranura $h(k)$ de la tabla HT ya está ocupada, el registro debe ser almacenado en otra ranura de la tabla.
- Será trabajo de la política de resolución de colisiones el determinar en qué otra ranura se debe almacenar el registro.
  - *Naturalmente, se debe seguir la misma estrategia de resolución de colisiones durante la búsqueda y durante la inserción, para ser capaces de encontrar el elemento.*

Hashing Cerrado

$$h(k_1) = h(k_2) = b$$



- Queremos insertar k_2, pero k_1 ocupa su ranura.
- Debemos insertar k_2 en una celda alternativa.

Hashing Cerrado

Cualquier método de resolución de colisiones puede verse como un generador de secuencias de ranuras $\langle r_0, r_1, r_2, \ldots \rangle$ *de la tabla que pueden (potencialmente) almacenar el registro* (k, I)

- La primera ranura r_0 en la secuencia siempre es $h(k)$.
- Si esa ranura ya ha sido ocupada por otro registro, se chequea en la siguiente ranura r_1 de la secuencia
- Si $r_1$está ocupada, se chequea r_2, y así hasta encontrar una ranura libre.
- La secuencia de ranuras a inspeccionar para la clave k será generada por una función p tal que:

$$r_0 = h(k)$$

$$r_i = r_0 + p(k, i) \text{ mod } M \ \forall i > 0$$

Hashing Cerrado

Probe Function

- Aquí quien se encarga de definir la política de resolución de colisiones es $p(k, i)$.
  - Llamada *probe function*.
  - Ésta se puede implementar de múltiples formas, pero la más común es usando otra función de hashing h_2
    - i.e., tendríamos una función de hashing para calcular la ranura inicial r_0 y otra para la política de resolución de colisiones.
    - Para que sirve esto?, para evitar distribuir lo más posible los datos sobre la tabla.

Hashing Cerrado

Factor de Carga

- La performance de una tabla de hashing está directamente relacionada a la cantidad de datos que hay almacenados, en comparación a la cantidad de ranuras libres de la tabla.
- Una buena métrica es a lo que se le llama factor de carga:

$$\alpha = \frac{n}{M}$$

  - Indica el porcentaje de utilización de la tabla
  - Factor cercano a 1 indica que la probabilidad de colisiones es alta
  - Factor cercano a 0 indica una baja probabilidad de colisiones
  - Un $\alpha \approx 0.6$ suele ser un buen balance
- Utilizando esto se puede demostrar que el tiempo de ***busqueda*** e ***inserción*** en una tabla de hashing es $O(1)$ en *promedio*.

Tablas de Hashing en Python

Tablas de Hashing en Python: Dict

- Las tablas de hashing están presentes en python.
- En python la clase `dict` está implementada usando tablas de hashing.
  - Utiliza ***hashing cerrado***, con:
    - Una política de resolución de colisiones basadas en hashing (se suele llamar doble hashing), y
    - Con un factor de carga, $\alpha = \frac{2}{3}$.
  - Además la clase set también utiliza tablas de hashing, pero ahora con: $k_j = I_j$.

Tablas de Hashing en Python: Dict

Operación	Tiempo Ejecución	Descripción
`M[k]`	*O(1)*	*Retorna el valor asociado a la clave k*
`M[k] = v`	*O(1)*	*Asocia la clave k al valor v*
`del M[k]`	*O(1)*	*Remueve el valor v con clave k*
`len(M)`	*O(1)*	*Retorna el número de items en M*
`k in M`	*O(1)*	*Retorna True si la clave k se encuentra en M*
`M.get(k, d=None)`	*O(1)*	*Retorna M[k] si la clave existe, en otro caso retorna el valor d por defecto.*
`M.setdefault(k, d)`	*O(1)*	*Si la clave k existe retorna M[k], en otro caso asigna M[k] = d y retorna d.*
`M.pop(k, d=None)`	*O(1)*	*Remueve el item asociado a la clave k y retorna su valor asociado, si la clave no existe retorna d.*
`M.popitem()`	*O(1)*	*Remueve un par (k, v) arbitrario y retorna dicho par.*

Tablas de Hashing en Python: Dict

Operación	Tiempo Ejecución	Descripción
`M.clear()`	*O(n)*	*Remueve todos los items*
`M.keys()`	*O(1)*	*Retorna todas las claves de M*
`M.values()`	*O(n)*	*Retorna todos los valores de M*
`M.items()`	*O(n)*	*Retorna todos los pares (k, v) en M*
`M.update(M2)`	$O(n_2)$	*Asigna M[k] = v para cada par en en M2*
`M == M2`	$O(\text{mín}\{n, n_2\})$	*Retorna True si cada par (k, v) es idéntico en M y M2.*
`M != M2`	$O(\text{mín}\{n, n_2\})$	*Retorna True si en M y M2 los pares (k, v) no son idénticos.*

Tablas de Hashing en Python: Set

Operación	Tiempo Ejecución	Descripción
`e in S`	*O(1)*	*Pertenencia: Retorna True si el elemento e existe en S.*
`S.add(e)`	*O(1)*	*Añade el elemento e. No tiene efecto si e ya está presente.*
`S.remove(e)`	*O(1)*	*Elimina e. Lanza KeyError si el elemento no existe.*
`S.discard(e)`	*O(1)*	*Elimina e si está presente. No lanza error si no existe.*
`S.pop()`	*O(1)*	*Elimina y retorna un elemento arbitrario. Lanza KeyError si está vacío.*
`len(S)`	*O(1)*	*Retorna la cantidad total de elementos en el conjunto.*
`S.clear()`	*O(n)*	*Remueve todos los elementos, dejando el conjunto vacío.*

Nota: estos tiempos son logrados usando tablas de hashing